

הדבר החשוב להבין ראשון: **בין בדיקה לבדיקה שום תהליך שלך לא רץ**. אין שרת שמאזין, אין לולאה שממתינה. Cloud Run מחזיק תמונת מכולה (container image) רדומה, ומעיר אותה רק כשמגיעה בקשת HTTP. כשהבקשה נגמרת — המכולה חוזרת לישון. זו הסיבה שאתה משלם רק על שניות עבודה בפועל, וזו גם הסיבה שצריך מישו חישובי שידפוק בדלת.

שרשרת ההערה — מי דופק בדלת

Cloud Scheduler

1

cron מנוהל

משימה בשם oris-scanner-poll עם לוח זמנים * * * * *. /5 זהו cron מנוהל של גוגל — הוא לא מריץ קוד, הוא רק שולח בקשת רשת בזמנים קבועים. אין לו מושג מה יש בצד השני.

↓ POST · גוף ריק · חתום ב־OIDC

שער הכניסה של Cloud Run

2

אימות

השירות פרוס עם --no-allow-unauthenticated , כלומר סגור לאינטרנט. הסקדיולר מצורף ל־ service account שקיבל את התפקיד roles/run.invoker , וחותרם כל בקשה בטוקן **OIDC** (OpenID Connect — שכבת זהות מעל OAuth 2.0: תעודה חתומה שאומרת "אני זה־זה", לעומת טוקן OAuth רגיל שאומר רק "מותר לי"). ה־ audience של הטוקן הוא כתובת השירות עצמה, כך שטוקן שדלף לא יעבוד מול שירות אחר.

↓ הבקשה אושרה

הרמת המכולה

3

cold start

אם אין מכולה חיה, Cloud Run מרים אחת: 1 vCPU, 512 MiB startup-cpu-boost דלוק כדי לקצר את ההתנעה. בפנים עולה functions-framework — עטיפה שמקימה שרת Flask ומנתבת כל בקשה לפונקציה אחת שהוגדרה בדיפלוי: --function=process_my_drive_files .

↓ קריאה לפונקציה

נקודת הכניסה

4

main.py:413

@functions_framework.http
def process_my_drive_files(request):
 try:
 run_workspace_scan()
 except Exception:
 log.exception("scan failed")
 raise
 return ("ok", 200)

ההפתעה: אף פרמטר לא מועבר בבקשה.
הארגומנט request מתקבל — ומיד מתעלמים ממנו לחלוטין. גוף ה־POST ריק. הבקשה היא **פעמון בלבד** : "התעורר וסרוק". היא לא נושאת מידע על תלמיד, על משימה או על קובץ. כל מה שהסורק צריך לדעת הוא מוצא בעצמו, ברגע שהוא מתעורר.

אז מאיפה כן מגיעה האינפורמציה?

מקור	נקבע איפה	מה מגיע דרכו
משתני סביבה	deploy.sh → --set-env-vars	LOG_LEVEL , AZURE_OPENAI_DEPLOYMENT , AZURE_OPENAI_ENDPOINT — נצרכים ברוויזיה, משתנים רק בדיפלוי חדש
Secret Manager	deploy.sh → --set-secrets	מפתחות ה־API של ג'מיני / אנתרופיק / גרוק / אז'ור. מוזרקים כמשתני סביבה בזמן ריצה — לא יושבים בקוד ולא ב־git
גוגל קלאסרום	ממשק המורה	הפרמטרים החיים : תיאור המשימה (=השאלה), תגית [model: ...] , המחווין, וקבצי ההגשה. נקראים מחדש בכל סריקה — המורה משנה התנהגות בלי שאף אחד נוגע בקוד

חיוב לפי בקשה

cpu-throttling בבירית מחדל. מחוץ לטיפול בבקשה ה־CPU חנוק ואינו מחויב. סריקה ריקה נמדדה ב־**6.0 שניות** — זה כל מה שמשלמים עליו.

למה לא Pub/Sub

התכנון המקורי עבר דרך Pub/Sub. הוא נדרש רק אם קלאסרום דוחף התראות (ה־Registrations API יודע לפנות לנושא Pub/Sub בלבד, לא ל־webhook). אנחנו מושכים ביוזמתנו, אז הקפיצה הזאת הוסרה — מערכת retry אחת במקום שתיים.

המכולה ערה, ואין לה שום מידע. עכשיו היא צריכה לגלות בעצמה אם משהו הגיש משהו. זה נעשה במשפך של סינונים — כל שלב מצמצם, וברוב המוחלט של הסריקות המשפך מתרוקן ולא קורה כלום.

שלב א' — להשיג זהות

main.py:206

התחזות מבוקרת

גוגל קלאסרום לא מדבר עם **service accounts** — רק עם משתמשים אמיתיים. לכן בכל סריקה נבנות הרשאות מואצלות (domain-wide delegation): המכולה חותמת דרך iam.Signer בשם sainter@... , ומבקשת טוקן עם "subject=ori@bdika.net". התוצאה — היא פונה לקלאסרום כאילו היא המורה, בארבעה scopes: קורסים, נושאים, עבודות תלמידים, ודרייב.

שלב ב' — המשפך

1 קריאה

1 כל הקורסים

courses.list(teacherId='me') — כל הקורסים שהמורה מלמד.



קריאה לכל קורס

2 סינון לפי נושא

topics.list — מחפש נושא בשם "English Assignment". קורס בלי הנושא הזה מדולג כולו. זו הסיבה השכיחה ביותר שהגשה לא נתפסת — ולכן היא נרשמת ללוג יחד עם רשימת הנושאים שכן קיימים.



קריאה לכל קורס תואם

3 משימות תחת הנושא

courseWork.list , ואז סינון ב־Python לפי topicId .



2 קריאות לכל משימה

4 הגדרות המשימה

rubrics.list — מחוון נייטיב של קלאסרום, אם קיים.
studentSubmissions.list — כל ההגשות.
ובמקביל, ללא רשת: resolve_model() שולף תגית [model: ...] מהתיאור, ו־ resolve_rubric_material() מזהה קובץ מצורף שעשוי להיות מחוון.



סינון

5 רק מה שהוגש באמת

עוברות רק הגשות במצב TURNED_IN או RETURNED . קובץ שהתלמיד צירף אך לא לחץ "הגש" כבר יושב בדרייב — אבל מצבו NEW / CREATED , והוא מדולג בלי לתבוע אותו, כדי שברגע שיגיש הסריקה הבאה תתפוס אותו כרגיל.



טרנזקציה

6 פנקס הכפילויות ב־Firestore

_try_claim_submission() — טרנזקציה אטומית על מסמך שמפתחו הוא מזהה ההגשה. שלוש תוצאות:

- לא קיים → נתבע, מסומן in_progress , ממשיכים לבדיקה.
- done עם אותו updateTime → כבר נבדק, דילוג.
- done עם updateTime שונה → התלמיד הגיש מחדש, נבדק שוב.

מזהה ההגשה יציב אצל קלאסרום גם אחרי unsubmit/resubmit, ולכן סימון "בוצע" לנצח היה משתיק הגשות חוזרות. הגרסה נשמרת יחד עם הסטטוס בדיוק כדי למנוע את זה.

המחיר של המשפך — 15 קריאות API לסריקה (נמדד)

מול תקרת הקלאסרום הכובלת של 600 בקשות לדקה לפרויקט למשתמש — כלומר 2.5% ממנה. שתי התקרות האחרות (3,000/דקה לפרויקט, 4 מיליון/יום) רחוקות עוד יותר.

נקודת שבירות שכדאי להכיר

אין ניסיון חוזר ברמת הקריאה הבודדת. שגיאת 503 אחת מקלאסרום מפילה את כל

הסריקה, כולל קורסים שטרם נסרקו. ב־28/07 היו שלוש כאלה. הסריקה הבאה מכסה על זה, אבל בקצב של 5 דקות הפער גדל.

כאן מתחיל החלק היקר — הקריאות למודלים. הוא רץ רק על הגשה שנתבעה בהצלחה, ולכן סריקה ריקה לא עולה כלום מעבר לשש השניות.

א' — להחליט לפי מה בודקים

checker.py:150

סדר עדיפויות המחונן

עדיפות	מקור	מתי
1	מחונן נייטיב של קלאסרום	הצהרת הכוונה המפורשת ביותר, וזה מה שהמורה רואה בממשק הציונים
2	מסמך מצורף שסווג כמחונן	שם הקובץ מכיל "מחונן"/ rubric , או תגית [rubric: ...] , או — קובץ מצורף יחיד שהמסווג קבע שהוא מחונן
3	המחונן המובנה esl-9th-grade-analytical	ברירת מחדל כשאין אף אחד מהשניים

מסמך מצורף שאינו נושא שם של מחונן נקרא בכל זאת, ועובר סיווג (core.classify_document , קריאת טקסט קצרה אחת): אם הוא מכיל קריטריונים וניקוד — הוא מחליף את המחונן המובנה. אם הוא הגדרת תרגיל — הוא **נצרף לשאלה כהקשר** והמחונן המובנה נשאר. הכיוון הזה מכוון: תוספת הקשר יכולה רק להועיל, בעוד מחונן שגוי מוחק את סולם הציונים.

ב' — קריאת הדפים

1 הורדה מדרייב

files.get לזיהוי ה־ MIME type , ואז הורדת הבתים. סוג לא נתמך מדולג עם רישום ללוג.

↓ לכל עמוד בנפרד

2 זיהוי סיבוב

detect_rotation — שולח את העמוד מוקטן ל־900px ושואל "כמה מעלות לסובב?". מחזיר 0/90/180/270. עוקב אחרי אותו model_key כמו שאר הציונים.

↓

3 OCR

סיבוב, הקטנה ל־ MAX_EDGE , ואז ocr_page — המרת כתב היד לרשימת שורות. במקביל נשמר original_b64 , תצוגה מוקטנת של העמוד המקורי, לשיבוץ בדוח.

↓ כל העמודים אוחדו לתמליל אחד

4 הערכה

evaluate_with_rubric — קריאה **אחת** על התמליל המאוחד, עם סכימת JSON מחייבת: ציון ורמה לכל קריטריון, משוב מילולי, וציטוטים נקודתיים לשורות בעייתיות. השאלה של המשימה נשתלת לתוך המחונן כשדה question , כדי שהמודל יעריך מול המשימה בפועל ולא רק מול קריטריונים גנריים. אצל ג'מיני מוגדר thinking_budget=0 — בלעדיו המודל שורף את תקציב הפלט על חשיבה פנימית וה־JSON נחתך באמצע.

ג' — הדוח והתיקייה

main.py:164

מציאת תיקיית היעד

התיקייה "תרגילים בדוקים" נוצרת פעם אחת לכל משימה, בתוך תיקיית ההגשות שקלאסרום כבר יצר — היא נמצאת דרך ההורה של הקובץ שהתלמיד הגיש. המיפוי folder → coursework נשמר ב־Firestore, כדי שכל הרצה תגיע לאותה תיקייה.

report.py

בניית ה־DOCX

build_evaluation_docx מרכיב מסמך Word: טבלת קריטריונים בצבעים, המשוב, ותמונת העמוד המקורי כשהציטוטים הבעייתיים מסומנים בהדגשה עליה. שם הקובץ: <חותמת זמן>_report_<שם המקור>.docx
ציטוט שהמודל החזיר ולא נמצא מילה־במילה בשורת ה־OCR מדולג ונרשם כאזורה — ההערה יורדת מהדוח, הציון לא נפגע.

העלאה, ואז סגירת המעגל

files.create עם MediaIoBaseUpload מעלה את ה־DOCX לתיקייה. מיד אחריו _mark_done(subm_id, update_time) רושם בפנקס שההגשה הזאת, **בגרסה הזאת**, טופלה — וזה מה שמונע בדיקה חוזרת בסריקה הבאה, בלי לחסום הגשה מחודשת אמיתית.

זה לא תרשים תיאורטי — זו הרצה שנרשמה ביומן. התלמיד הגיש ב־15:24:33, הסריקה הבאה תפסה אותה, והדוח היה בתיקייה 56 שניות אחר כך.

15:24:33	התלמיד לוחץ "הגש" — המצב עובר ל־TURNED_IN, ה־updateTime מתעדכן
ישנ 33 ↓	המתנה לפעיימת הסקדיולר הבאה
15:25:06	זוהתה — המשפך עבר, הפנקס תבע את ההגשה
15:25:07	המחונן נקרא — 3,967 תווים משכבת הטקסט של ה־PDF. בלי OCR, בלי עלות
15:25:08	נוצרה תיקיית "תרגילים בדוקים" — פעם ראשונה למשימה הזאת
15:25:10	הורדת ההגשה — dan sarig.pdf, 870 KB
15:25:14	קריאה 1 — זיהוי סיבוב (≈1 שני')
15:25:15	קריאה 2 — 5 — OCR שניות, 17 שורות חולצו
15:25:20	קריאה 3 — הערכה מול המחונן — 7 שניות, 4 קריטריונים
15:25:27	תוצאה: A2 to B1 · 1 עמוד · 112 מילים · הבדיקה ארכה 17.5 שני'
15:25:29	הדוח הועלה לתיקייה — dan sarig_report_20260728-152527.docx · סה"כ 20.8 שני'
15:25:34	הסריקה הסתיימה (31.0 שני') — המכולה חוזרת לישון

המספרים שמאחורי המערכת

עלות חודשית ב־*/5		סריקה ריקה (הרוב המוחלט)	
בקשות	2M / 8,766 חינם	משך	6.0 שני'
vCPU-שניות	~52,600 / 180K חינם	קריאות קלאסרום	15
GiB-שניות	~26,300 / 360K חינם	קריאות מודל	0
סה"כ Cloud Run	\$0	סריקה עם הגשה אחת	
מכסת החינם משותפת עם checker/scan1/scan2 — אם הם יבלעו אותה, ~\$1.3 לחודש		משך	31 שני'
		קריאות מודל	3 לעמוד־ועוד־הערכה

שלוש נקודות שכדאי לזכור על Cloud Run כאן

- אין מצב (state) בין בקשות. כל מה שצריך לשרוד — מי כבר נבדק, איפה התיקייה — יושב ב־Firestore, לא בזיכרון. שתי מכולות יכולות לרוץ במקביל.
- הרוויזיה היא היחידה הפרוסה. `--set-env-vars` מחליף את כל בלוק הסביבה, ולכן שינוי ידני דרך `services update` נמחק בדיפלווי הבא — בדיוק הסיבה ש־LOG_LEVEL נעוץ ב־`deploy.sh`.
- הסקדיולר אינו חלק מהדיפלווי. `deploy.sh` מעלה קוד בלבד; חיווט הטריגר יושב ב־`setup_infra.sh` ורץ בנפרד.